

Arquitectura de Computadoras

TP7 - GPU

Introducción	1
Motivación	1
La GPU	2
Herramientas de trabajo	2
Ejercicios	3
Ejercicio 1 – Hola mundo.....	3
Ejercicio 2 – Modos de programación	3
Ejercicio 3 – Divide y vencerás.....	4
Ejercicio 4 – Manos a la obra	12
Ejercicio 5 – Aplicaciones del procesamiento mediante GPU.....	12

Introducción

Motivación

A lo largo de la materia, introducimos el concepto de CPU como la unidad de procesamiento de la computadora, encargada de interpretar y ejecutar las instrucciones de software que componen al programa en ejecución. Si bien hasta hace poco la consideramos mono núcleo y, en consecuencia, capaz de procesar una sola instrucción a la vez, recientemente introducimos el concepto de procesadores multi núcleo, que permiten ejecutar más de una instrucción en simultáneo.

Para entender qué es una GPU y qué la hace tan útil, pongamos nuestro foco en este punto: es posible para una computadora moderna ejecutar más de un conjunto de instrucciones en simultáneo. Naturalmente, y para evitar tener que sincronizar los distintos núcleos, estos correrán bloques de código independientes uno del otro; es decir, un programa con lógica lineal tradicional, de los que han escrito cientos, ejecutado en una PC multi núcleo, correrá en uno solo, que se le asignará, sin dividirse y sin introducir ninguna rareza. Cada bloque de instrucciones, corriendo en cada núcleo, tendrá siempre su propio Stack, su IP, y todos los recursos a los que estamos acostumbrados. En consecuencia, la utilidad más evidente para multi núcleo es correr distintos programas independientes, uno en cada núcleo. La distribución de los programas en los distintos núcleos ya se encarga, sin que lo sepamos, el Sistema Operativo. Entonces, ¿de qué sirve saber todo esto, más allá de entenderlo como una optimización que aprovecha el Sistema Operativo?

La respuesta es que, efectivamente, como programadores podemos aprovechar la existencia de varios núcleos para acelerar flujos de procesamiento paralelos. A lo largo de la siguiente guía exploraremos ejemplos de problemas paralelizables, y entenderemos cómo puede la GPU ayudarnos a resolverlos de forma eficiente.

La GPU

¿Qué es, entonces, una GPU? La sigla significa Graphics Processing Unit (pues suele usarse, aunque no de manera exclusiva, para lidiar con problemáticas relacionadas a los gráficos, como la renderización de un videojuego), y se trata de una unidad de procesamiento que se comporta de forma similar a un procesador, en el sentido de que ejecuta código, aunque con un set de instrucciones diferente, y una capacidad gigantesca para ejecutar hilos de código simultáneos. Supongan que se les pide escribir un algoritmo que, dadas las listas de notas de todos los alumnos universitarios de la Argentina, calcule la nota más baja de cada alumno.

La solución es trivial: por cada alumno, se recorre cada una de sus calificaciones y se encuentra el menor valor, todo de manera secuencial. Ahora, esta solución crecerá, en tiempo de ejecución, linealmente con la cantidad de alumnos, puesto que la secuencialidad del algoritmo implica que solo después de calcular el valor para un alumno, se procederá a hacerlo con el siguiente.

Ahora, la pregunta que surge es: ¿por qué hacerlo de manera secuencial en lo absoluto? Asumiendo que sabemos cómo y qué disponemos de un procesador multinúcleo, ¿qué nos impide calcular la nota más baja de varios alumnos en simultáneo? Resulta que nada, porque no requerimos de los resultados del algoritmo para un registro anterior para calcular otro cualquiera. Este es un caso de uso ideal para programación concurrente: podríamos tener varios núcleos de procesamiento calculando, de manera independiente y simultánea, la nota más baja de varios alumnos.

Un Intel Core i7, por ejemplo, puede tener cuatro núcleos, por lo que, idealmente, podría correr 4 hilos que calculen la nota más baja de 4 alumnos simultáneamente, y repetir este proceso hasta acabar con todos. Esto ya reduciría el tiempo de procesamiento total sensiblemente.

Las GPU existen específicamente para lidiar con este tipo de problemas, y ponen a disposición del programador no ya 2, 5 o 7 hilos simultáneos, sino miles. Una GPU competente podría calcular la nota más baja para miles de alumnos en simultáneo, reduciendo drásticamente el tiempo de procesamiento.

Herramientas de trabajo

Para facilitarles encarar los siguientes ejercicios, la cátedra pone a su disposición un equipo con una poderosa GPU para que utilicen. El mismo es accesible desde la red interna del ITBA, por lo que será visible desde pampero. Su nombre es "titan-arq", y tiene habilitado el acceso por SSH para que ingresen con sus credenciales actuales. Cabe destacar que el equipo tiene ya instaladas todas las herramientas que necesitarán para encarar la guía.

Desde su terminal, ingresen a pampero con sus credenciales:

```
$ ssh miusuario@pampero.itba.edu.ar
```

Una vez dentro de pampero, ingresen a la PC "arquitectura" mediante SSH, con las mismas credenciales:

```
$ ssh arquitectura -L8080:localhost:8080
```

Ejercicios

Ejercicio 1 – Hola mundo

Así como Intel y AMD utilizan la arquitectura 8086, y ARM la propia, las GPU de Nvidia utilizan una llamada CUDA. Para compilar código al lenguaje ensamblador de CUDA, Nvidia ofrece un paquete llamado CUDA Toolkit¹, que contiene un compilador y un analizador de performance, entre otras herramientas.

El compilador, `nvcc`, es muy similar a `gcc`, pero trabaja con una variante de C++ que amplía su sintaxis con algunas anotaciones útiles que ya discutiremos. La extensión esperada de los archivos de código es `.cu`, y él mismo genera ejecutables que pueden correrse como cualquier programa normal.

Por lo pronto, es importante mencionar que todo programa C válido es también un programa C++ válido, por lo que todo lo que saben de C, como el uso de headers, sintaxis de funciones y librería estándar, sigue valiendo. Salvo por las anotaciones específicas de GPU que discutiremos a su debido tiempo, pueden trabajar como si estuviesen en C. También, cabe destacar que a todo el código que veremos en esta guía lo compilaremos con `nvcc`.

Consigna

Escriban un programa CUDA que imprima el mensaje “¡Hola, mundo!” por consola, compílenlo con `nvcc` y ejecútenlo.

Preguntas adicionales

- ¿El ejecutable generado puede correr en cualquier PC?
- ¿Es posible depurar este programa con las herramientas que conocemos?

Ejercicio 2 – Modos de programación

Así cómo es posible, mediante las system calls, comunicarse y hacer pedidos al Sistema Operativo, la GPU también ofrece no una, sino dos interfaces de comunicación que permiten al programador dialogar con el dispositivo. Las mismas son la Runtime API² y la Driver API³. Ambas pueden incluirse como una librería en un archivo de código, y exponen distintas funciones que pueden invocarse para interactuar con la GPU. En la mayor parte de los casos, la Runtime API y la Driver API pueden usarse de modo intercambiable.

Nota: Para incluir funciones de la Runtime API o de la Driver API, deben agregar, a la llamada a `nvcc`, el flag `-lcuda`, para asegurar que el compilador linkeará contra las librerías de CUDA.

Consigna

Escriban dos programas que impriman en pantalla la cantidad de GPUs disponibles y, por cada una, la cantidad de multiprocesadores con los que cuentan, y la memoria total de la que disponen (en MB), utilizando primero la Runtime API y luego la Driver API. Una pista:

¹ <https://developer.nvidia.com/cuda-toolkit>

² <https://docs.nvidia.com/cuda/cuda-runtime-api/>

³ <https://docs.nvidia.com/cuda/cuda-driver-api/>

¡busquen funciones útiles en la documentación de la Runtime API y la Driver API según corresponda!

Preguntas adicionales

- Si pueden usarse de forma intercambiable, ¿por qué la Runtime API y la Driver API existen como entidades independientes? ¿Existen casos en los que se recomienda usar una y no la otra?

Ejercicio 3 – Divide y vencerás

Como discutimos anteriormente, lo más interesante de las GPUs es su capacidad para correr miles de hilos de código en simultáneo. Esto resulta especialmente útil a la hora de encarar problemas altamente paralelizables.

Un ejemplo de esto es el dibujado en pantalla de, por ejemplo, un videojuego: distintas partes de esta pueden dibujarse simultáneamente; entonces, puede asignarse a hilos distintos la tarea de dibujar distintas áreas, y así agilizar sensiblemente el proceso. Esto explica por qué las computadoras modernas pueden dibujar a enormes velocidades, mientras que el dibujado lineal monohilo que implementaron en el TPE de la materia resulta tan lento. Para terminar de entender este concepto, les recomendamos el siguiente vídeo:

<https://www.youtube.com/watch?v=-P28LKWTzrl>

Para llevar estos conceptos a la práctica, consideremos el siguiente problema: “Dados dos vectores de enteros de idéntica longitud, generar un tercer vector que contenga la suma de los elementos componente a componente”.

El código que lo resuelve, programado para correr en la CPU en un solo hilo, podría verse así.

```
//sum_cpu.cu

#include <stdio.h>
#include <stdlib.h>
#include <time.h>

// Cantidad de elementos en el array. Disminuir si no alcanza la memoria
#define ARR_SIZE 100000000

// Suma dos arrays, nums1 y nums2, componente a componente, en nums3
void sum_arrays(int *nums1, int *nums2, int *nums3, int dimension) {
    for (int i = 0; i < dimension; i++)
        nums3[i] = nums1[i] + nums2[i];
}

// Evalúa posibles errores
int check_error(int *nums1, int *nums2, int *nums3, int dimension) {
    for (int i = 0; i < dimension; i++)
        if (nums1[i] + nums2[i] != nums3[i])
            return -1;
}
```

```

    return 0;
}

int main() {
    // Reservo espacio para 3 vectores de enteros con ARR_SIZE elementos
    int *nums1, *nums2, *nums3;
    nums1 = (int *)malloc(ARR_SIZE * sizeof(int));
    nums2 = (int *)malloc(ARR_SIZE * sizeof(int));
    nums3 = (int *)malloc(ARR_SIZE * sizeof(int));

    // Rellenamos los arrays con valores arbitrarios
    for (int i = 0; i < ARR_SIZE; i++) {
        nums1[i] = 1;
        nums2[i] = 2;
    }

    // Guardamos la hora actual
    clock_t start = clock();

    // Ejecutamos el cálculo
    sum_arrays(nums1, nums2, nums3, ARR_SIZE);
    clock_t end = clock();
    double time_taken = ((double)(end - start)) / (CLOCKS_PER_SEC / 1000);
    printf("Resolver el problema (excluyendo inicializaciones) llevo %f\n", time_taken);

    if (check_error(nums1, nums2, nums3, ARR_SIZE) != 0)
        printf("Resultado incorrecto\n");
    else
        printf("Resultado correcto\n");

    // Libero los recursos
    free(nums1);
    free(nums2);
    free(nums3);
}

```

Podrán ver que es código C válido, por lo tanto, también es código C++ válido. Compilémoslo con nvcc, y veamos qué ocurre.

Nótese que computamos la hora de inicio y fin de la función `sum_arrays` para independizarnos de los tiempos de inicialización y reserva de memoria, y poder así evaluar con mayor precisión la performance del algoritmo.

Ahora, este algoritmo es lineal: debe computar la suma del i -ésimo elemento antes de computar la del siguiente.

Vamos a intentar modificarlo para sacarle provecho a la GPU, pero primero, debemos entender un concepto importante: todo el código que escribimos hasta ahora, incluso si se compila con nvcc, corre en la CPU, como cualquier programa normal. Es decir, en ningún momento le pedimos a la GPU que haga nada, por lo que el ejecutable generado por nvcc para este código es virtualmente equivalente al que se obtiene compilándolo con gcc.

¿Cómo hacer que una función corra, en cambio, en la GPU?

Nvcc define la etiqueta global que, aplicada a una función que debe devolver void, correrá en la GPU y no en la CPU si se la invoca apropiadamente. Esta función deberá ser invocada utilizando una sintaxis especial, que incluye, entre picos, el número de hilos que se crearán, aunque a los efectos de este ejemplo, crearemos uno solo.

Además, es importante recordar que la GPU es un dispositivo independiente, es decir, que tiene sus propias componentes, como caché, registros, y, fundamentalmente, su propia memoria, totalmente independiente de la RAM que utilizamos desde la CPU; esto se traduce en que no podemos generar un puntero desde la CPU (por ejemplo, con malloc, o pasando como parámetro la dirección de memoria de una variable local) y referenciarlo desde la GPU, puesto que las memorias RAM y de vídeo son totalmente independientes. Existe una función equivalente a malloc, llamada cudaMalloc, que reserva espacio en la memoria de vídeo, otra equivalente a free llamada cudaFree, y una llamada cudaMemcpy, que permite copiar información de una memoria a la otra, y así transferir la información para procesarla donde corresponda. Más recientemente, Nvidia implementó la función cudaMallocManaged, que facilita la gestión de memoria generando un único puntero válido tanto en la CPU como en la GPU, aunque sacrificando control sobre el copiado de memoria. La utilizaremos por simplicidad.

Por último, debe contemplarse que la creación de hilos en la GPU es asíncrona: cuando ordeno a la GPU que corra una función, desde la perspectiva de la CPU, la misma retorna automáticamente, y no existe garantía de que haya aún terminado. Como nosotros queremos aguardar a que el cómputo termine antes de verificar el resultado, debemos poner a la CPU en espera hasta que esto ocurra. Esto lo hacemos mediante la función cudaDeviceSynchronize, que congela el código de la CPU hasta que todos los hilos de la GPU hayan concluido su procesamiento.

Modificando ligeramente nuestro código para contemplar estas salvedades, obtenemos:

```
//sum_gpu_singleThread.cu

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <cuda_runtime.h>

// Cantidad de elementos en el array. Disminuir si no alcanza la memoria
#define ARR_SIZE 100000000

// Suma dos arrays, nums1 y nums2, componente a componente, en nums3
// Atributo "global" indica que función correrá en la GPU. ¡Debe devolver void!
__global__
```

```

void sum_arrays(int* nums1, int* nums2, int* nums3, int dimension) {
    for (int i = 0; i < dimension; i++)
        nums3[i] = nums1[i] + nums2[i];
}

// Evalúa posibles errores
int check_error(int* nums1, int* nums2, int* nums3, int dimension) {
    for (int i = 0; i < dimension; i++)
        if (nums1[i] + nums2[i] != nums3[i]) return -1;

    return 0;
}

int main() {
    int* nums1, * nums2, * nums3;
    // Reserva espacio para 3 vectores de enteros con ARR_SIZE elementos
    // Nótese que ya no utilizo malloc, porque malloc reserva memoria RAM
    // de la CPU, no accesible por la GPU
    // cudaMallocManaged permite reservar memoria tanto en la CPU como en
    // la GPU y se encarga de sincronizarlas para ahorrar nos esfuerzo
    // Es posible reservar memoria solo en la GPU, o incluso manejar toda
    // la sincronización manualmente si se desea, utilizando cudaMalloc,
    // cudaMemcpy y cudaFree
    cudaMallocManaged(&nums1, ARR_SIZE * sizeof(int));
    cudaMallocManaged(&nums2, ARR_SIZE * sizeof(int));
    cudaMallocManaged(&nums3, ARR_SIZE * sizeof(int));

    // Rellenamos los arrays con valores arbitrarios
    for (int i = 0; i < ARR_SIZE; i++) {
        nums1[i] = 1;
        nums2[i] = 2;
    }

    // Guardamos la hora actual
    clock_t start = clock();

    // Ejecutamos el cálculo
    // Valores entre picos indican cantidad bloques e hilos a crearse, se
    // discutirá en la próxima sección. En este caso, se usará un solo hilo.
    sum_arrays<<<1, 1>>>(nums1, nums2, nums3, ARR_SIZE);

    // Llamada a la GPU es asíncrona, es decir, retorna inmediatamente,
    // sin que la función haya necesariamente concluido su ejecución
    // Por este motivo, debemos esperar a que la GPU termine de trabajar
    // antes de continuar.
    cudaDeviceSynchronize();
}

```

```

    clock_t end = clock();
    double time_taken = ((double)(end - start)) / (CLOCKS_PER_SEC / 1000);
    printf("Resolver el problema (excluyendo inicializaciones) llevó %f
milisegundos\n", time_taken);

    if (check_error(nums1, nums2, nums3, ARR_SIZE) != 0)
        printf("Resultado incorrecto\n");
    else
        printf("Resultado correcto\n");

    // Libero los recursos
    cudaFree(nums1);
    cudaFree(nums2);
    cudaFree(nums3);
}

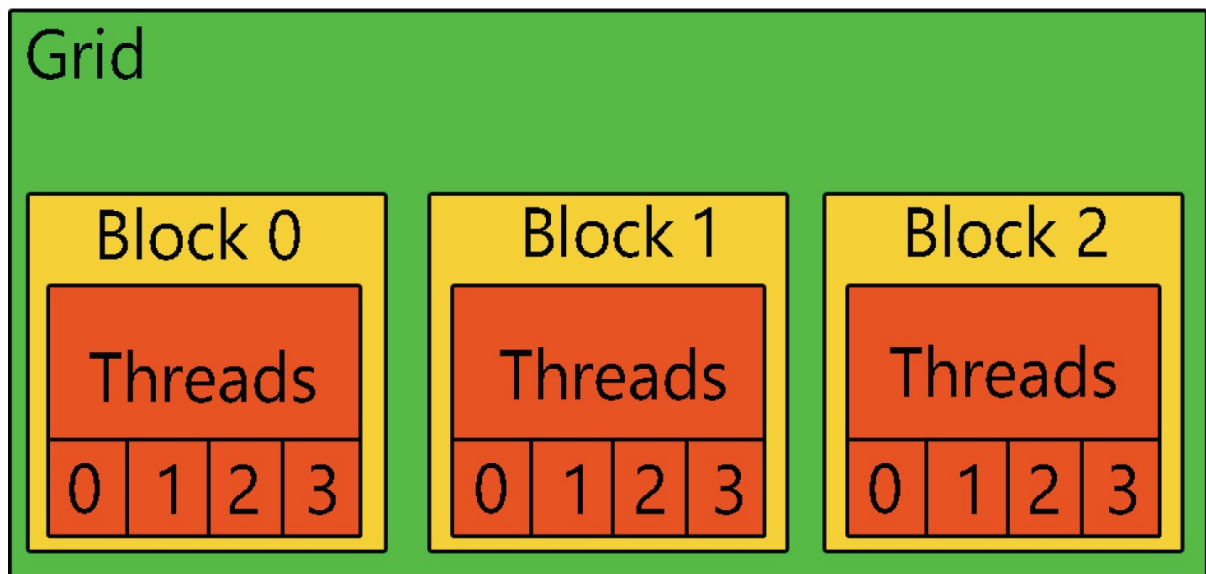
```

Naturalmente, notarán que no hemos aún aprovechado la capacidad multihilo de la CPU, y eso es lo que haremos a continuación.

Ya decidimos que nuestro problema es paralelizable, por lo que la próxima pregunta resulta: ¿qué tanto debo paralelizarlo? Podríamos crear dos hilos, y que cada uno procese la mitad de los elementos. Podemos crear 4 y que cada uno procese 1/4 de los elementos. Esto es lo que haríamos en una CPU, donde tenemos pocos núcleos y, en consecuencia, podemos correr solo unos cuantos hilos en simultáneo. Pero la GPU tiene la capacidad de procesar miles de hilos, entonces, ¿por qué no asignar cada elemento del arreglo a un hilo, y que se realicen miles de sumas simultáneas?

Siendo que ya definimos que crearemos un hilo por operación de suma (es decir, uno por elemento), queda ver cómo hacer esto, lo que requiere un entendimiento del esquema de grilla de la GPU. Los hilos en una GPU efectivamente se organizan dentro de una grilla, que puede ser de una, dos o tres dimensiones. Como nuestro problema es de una sola dimensión, utilizaremos una grilla de una sola dimensión, como la ilustrada por el esquema expuesto a continuación.

1D Grid of 1D Blocks



Como puede apreciarse en el dibujo, un hilo dado está ubicado dentro de un bloque, que a su vez está dispuesto dentro de la grilla. Si la ilustración (de tres bloques y 4 hilos por bloque) correspondiese a nuestra GPU, y nosotros quisiésemos sumar los elementos de dos vectores de 8 elementos (necesitándose entonces 8 hilos), se nos serían asignados los bloques completos 0 y 1.

Queda otra pregunta por responder, y es, dado que en todos los hilos corre la misma función, ¿cómo sabe cada uno cuál de las posiciones de los vectores le corresponde para sumar? La respuesta yace en la disposición en grilla de los hilos: un hilo dado puede saber a qué bloque corresponde, qué número de hilo es dentro del bloque, y las dimensiones de la grilla en general. Empieza entonces a cobrar sentido esta geometría extraña: puedo determinar el elemento mediante la fórmula:

$$\text{Elem} = (\text{blockIdx} * \text{blockDim}) + \text{threadIdx}$$

En nuestro ejemplo, si existen 8 elementos, y por lo tanto 8 hilos, y sé que soy el hilo 1 del bloque 1, y que la dimensión de cada bloque es 4, entonces el elemento que me toca procesar es el de posición $(1 * 4) + 1$, el quinto elemento.

Estas variables, `blockIdx`, `blockDim` y `threadIdx` están ya definidas para toda función que corra en la GPU y pueden consultarse en cualquier momento. Cada una de ellas tiene un atributo por cada eje cartesiano (x, y, z) debido a que, como ya establecimos, la grilla puede ser 2D o 3D de requerirse, aunque en este ejemplo estemos recurriendo a utilizar una sola dimensión. Por eso, notarán que, en el código, el número de elemento a procesar se obtiene utilizando solo la componente x de las variables:

$$\text{Elem} = (\text{blockIdx.x} * \text{blockDim.x}) + \text{threadIdx.x}$$

Aclarada la cuestión teórica, queda ver cómo indicarle a la GPU la cantidad de hilos a lanzar. En el ejemplo anterior, vimos que, para correr una función en la GPU, debía etiquetarse con la expresión global, y su invocación debía incluir el operador pico. Ese operador pico contiene dos parámetros: la cantidad de bloques que queremos reservar, y el número de hilos que queremos lanzar en cada bloque.

NomFuncion<<<NUM_BLOCKS, NUM_THREADS>>>(Argumento1, ..., ArgumentoN);

El número máximo de hilos por bloque depende de la GPU y puede consultarse de forma similar a como consultaron, en el ejercicio anterior, el nombre del dispositivo o su memoria. Es conveniente elegir un múltiplo de 32 por cuestiones de optimización en las que no ahondaremos (para los interesados, consultar el siguiente enlace y el concepto de warp); a los efectos de nuestro problema, elegiremos NUM_THREADS = 256 hilos por bloque.

Entonces, ¿cuántos bloques necesitaremos? Si queremos un hilo por elemento del vector, y tenemos 256 hilos por bloque, la cantidad puede calcularse de la siguiente manera:

NUM_BLOCKS = Techo(ARR_SIZE / THREADS)

Por supuesto, con esta fórmula, es posible que terminemos con más hilos que elementos en los vectores, por lo que debe considerarse este escenario para evitar procesar un elemento que no existe.

Finalmente, nuestro código terminado resulta:

```
//sum_gpu_multiThread.cu

#include <stdio.h>
#include <stdlib.h>
#include <time.h>
#include <math.h>
// Cantidad de elementos en el array. Disminuir si no alcanza la memoria.
#define ARR_SIZE 100000000
// Cantidad de hilos
#define THREAD_COUNT 256
// Suma una posición de nums1 y nums2, en nums3
// Atributo "__global__" indica que función correrá en la GPU. ¡Debe devolver void!
// blockDim, blockIdx, y threadIdx son definidas por la GPU y accesibles desde cualquier código que corra en la GPU, indican la posición del hilo actual en la grilla
// Utilizamos solo la componente x puesto que decidimos trabajar con una grilla de una dimensión.
__global__
void sum_arrays(int * nums1, int * nums2, int * nums3, int dimension) {
    int tid = (blockDim.x * blockIdx.x) + threadIdx.x;
    // Como redondeé para arriba el número de hilos, me aseguro que no sea uno de los que sobra
    if (tid < dimension)
        nums3[tid] = nums1[tid] + nums2[tid];
}
// Evalúa posibles errores
int check_error(int * nums1, int * nums2, int * nums3, int dimension) {
    for (int i = 0; i < dimension; i++)
        if (nums1[i] + nums2[i] != nums3[i])
```

```

        return -1;
    return 0;
}
int main() {
    // Genera dos arrays de enteros y lo rellena con numeros al azar
    int * nums1, * nums2, * nums3;
    cudaMallocManaged(&nums1, ARR_SIZE * sizeof(int));
    cudaMallocManaged(&nums2, ARR_SIZE * sizeof(int));
    cudaMallocManaged(&nums3, ARR_SIZE * sizeof(int));
    // Rellenamos los arrays con valores arbitrarios
    for (int i = 0; i < ARR_SIZE; i++) {
        nums1[i] = 1;
        nums2[i] = 2;
    }
    // Guardamos la hora actual
    clock_t start = clock();
    // Si cada bloque tendrá THREAD_COUNT hilos, ¿cuántos bloques
    necesitamos para tener un hilo por elemento del vector?
    int NUM_BLOCKS = (int)ceil(ARR_SIZE / THREAD_COUNT);
    // Corremos la función en NUM_BLOCKS bloques de THREAD_COUNT hilos
    cada uno
    sum_arrays<<<NUM_BLOCKS,    THREAD_COUNT>>>(nums1,    nums2,    nums3,
ARR_SIZE);
    // Llamada a la GPU es asíncrona, es decir, retorna inmediatamente,
    sin que la llamada a la función haya realmente concluído
    // Por este motivo, debemos esperar a que la GPU termine de trabajar
    antes de continuar.
    cudaDeviceSynchronize();
    clock_t end = clock();
    double time_taken = ((double)(end - start)) / (CLOCKS_PER_SEC / 1000);
    printf("Resolver el problema (excluyendo inicializaciones) llevó %f
milisegundos\n", time_taken);
    if (check_error(nums1, nums2, nums3, ARR_SIZE) != 0)
        printf("Resultado incorrecto\n");
    else
        printf("Resultado correcto\n");
    // Libero los recursos
    cudaFree(nums1);
    cudaFree(nums2);
    cudaFree(nums3);
}

```

Consigna

Compilen y corran las tres versiones del programa, y comparen los tiempos de ejecución para cada caso. ¿Qué pueden concluir?

Preguntas adicionales

- Investigen la herramienta nvprof de análisis de performance. ¿A qué corresponde el tiempo invertido en GPU page fault groups? ¿Cómo podría mejorarse?
- Modifiquen el código del ejercicio 2 para incluir el número total de hilos que la GPU puede computar en simultáneo.
- ¿Qué ocurriría si le pido a la GPU un número de hilos superior al valor obtenido en el punto anterior?

Ejercicio 4 – Manos a la obra

Con todo lo que aprendimos, es hora de que desarrollen, por su cuenta, su primer programa CUDA.

Implementen un programa que, dado un arreglo de enteros de tamaño arbitrario, calcule la suma de todos sus elementos, aprovechando el poder de paralelización de la GPU. Discutan sus implementaciones y decidan cuál es la mejor.

Ejercicio 5 – Aplicaciones del procesamiento mediante GPU

Propongan distintos problemas y evalúen su potencial de paralelización. Siendo que ya mencionamos el dibujado en pantalla, ¿en qué otras problemáticas conocidas podría resultar útil el procesamiento mediante GPU?